

■■■ SISTEMA DE GESTÃO DE OFICINA MECÂNICA

Documentação Técnica Completa

v3.0 — Portfólio Profissional

Spring Boot • MySQL • JWT • JUnit • Swagger • SonarQube • JaCoCo

Data: June de 2026

1. Controle de Versões

Versão	Data	Responsável	Descrição
v1.0	01/01/2025	Autor	Versão inicial — escopo e TAP
v2.0	01/03/2025	Autor	Requisitos funcionais e regras de negócio
v3.0	14/06/2026	Autor	Documento completo — todos os módulos

2. Sumário

1. Controle de Versões	2
2. Sumário	3
3. Termo de Abertura do Projeto (TAP)	4
4. Escopo Detalhado	5
5. Requisitos Funcionais (RF01–RF40)	7
6. Requisitos Não Funcionais (RNF)	10
7. Regras de Negócio (RN)	11
8. Perfis de Acesso	13
9. Fluxos de Negócio	14
10. Modelo Conceitual das Entidades	16
11. Modelo de Banco de Dados	18
12. Estrutura das APIs REST	20
13. Estrutura do Projeto Spring Boot	23
14. Tecnologias Utilizadas	25
15. Estratégia de Testes	27
16. Diagrama de Casos de Uso	29
17. Roadmap de Sprints	31
18. Plano de Backup e Recuperação	34
19. Glossário	36

3. Termo de Abertura do Projeto (TAP)

Nome do Projeto	Sistema de Gestão de Oficina Mecânica
Versão	3.0
Patrocinador	Portfólio Profissional Próprio
Gerente do Projeto	Desenvolvedor
Data de Início	01/01/2025
Previsão de Término	01/07/2025
Horas Estimadas	364 horas
Metodologia	Scrum (Sprints de 2 semanas)
Linguagem	Java 17+
Framework	Spring Boot 3.x
Banco de Dados	MySQL 8.x
Autenticação	JWT + Spring Security

3.1 Objetivo do Projeto

Desenvolver um sistema web completo para gestão de oficinas mecânicas, abrangendo cadastro de clientes e veículos, controle de estoque, gestão de ordens de serviço, faturamento, relatórios gerenciais e auditoria de operações. O sistema visa substituir processos manuais e planilhas por uma solução digital robusta, segura e escalável.

3.2 Justificativa

A gestão manual de oficinas mecânicas gera retrabalho, perda de informações, dificuldade no controle de estoque e falta de visibilidade financeira. Um sistema integrado reduz erros operacionais, melhora o atendimento ao cliente e fornece dados gerenciais para tomada de decisão estratégica.

3.3 Premissas e Restrições

Premissas:

- O desenvolvedor possui conhecimento em Java e Spring Boot
- O ambiente de produção utilizará servidor Linux com Java 17+
- O banco de dados MySQL 8.x estará disponível em servidor dedicado
- Todas as integrações externas serão via API REST
- O projeto seguirá boas práticas de Clean Code e SOLID

Restrições:

- Desenvolvimento por desenvolvedor solo (364h estimadas)
- Sem integração com sistemas fiscais reais na v1.0

- Interface web — aplicativo mobile fora do escopo atual
- Orçamento zero — ferramentas open source

4. Escopo Detalhado

Módulo 1: Segurança e Autenticação

- Cadastro, edição e desativação de usuários do sistema
- Login com geração de token JWT (Access + Refresh Token)
- Controle de perfis: ADMIN, GERENTE, ATENDENTE, MECÂNICO
- Criptografia de senha com BCrypt
- Auditoria de acessos e tentativas de login

Módulo 2: Cadastro de Clientes

- Cadastro completo: CPF/CNPJ, endereço, telefone, e-mail
- Vinculação de múltiplos veículos por cliente
- Histórico de atendimentos por cliente
- Busca por nome, CPF/CNPJ, placa do veículo
- Soft-delete (inativação sem exclusão física)

Módulo 3: Cadastro de Veículos

- Dados completos: placa, marca, modelo, ano, cor, chassi
- Histórico de ordens de serviço por veículo
- Controle de quilometragem por atendimento
- Vinculação com cliente proprietário

Módulo 4: Cadastro de Mecânicos

- Dados pessoais e profissionais do mecânico
- Especialidades (motor, elétrica, suspensão, funilaria)
- Histórico de ordens de serviço executadas
- Controle de produtividade por mecânico

Módulo 5: Fornecedores e Produtos

- Cadastro de fornecedores com CNPJ, contato, endereço
- Catálogo de produtos com código, descrição, categoria
- Preço de custo e venda por produto
- Vinculação produto-fornecedor

Módulo 6: Estoque

- Controle de entrada via Nota Fiscal
- Baixa automática ao utilizar produto em OS
- Alertas de estoque mínimo
- Relatório de movimentação de estoque
- Inventário com valor total em estoque

Módulo 7: Orçamento

- Criação de orçamento vinculado a cliente e veículo
- Adição de peças e serviços ao orçamento
- Aprovação ou rejeição pelo cliente
- Conversão de orçamento aprovado em OS

Módulo 8: Ordem de Serviço (OS)

- Geração automática a partir do orçamento
- Atribuição de mecânico responsável
- Status: Aberta, Em Andamento, Concluída, Cancelada
- Registro de peças utilizadas e serviços executados
- Controle de tempo de execução

Módulo 9: Faturamento

- Geração de Nota Fiscal de Saída (NFS)
- Emissão de recibo ao cliente
- Relatório financeiro mensal/anual
- Controle de formas de pagamento
- Dashboard de receitas

Módulo 10: Auditoria e Backup

- Log de todas as operações com usuário e timestamp
- Backup automático agendado do banco de dados
- Restauração de backup
- Relatório de auditoria por período

5. Requisitos Funcionais (RF01–RF40)

ID	Prioridade	Descrição
RF01	ALTA	O sistema deve permitir cadastro de usuários com perfil e senha criptografada
RF02	ALTA	O sistema deve autenticar usuários via JWT com refresh token
RF03	ALTA	O sistema deve controlar acesso por perfil (ADMIN, GERENTE, ATENDENTE, MECÂNICO)
RF04	ALTA	O sistema deve registrar log de login e logout de usuários
RF05	ALTA	O sistema deve permitir cadastro completo de clientes (PF e PJ)
RF06	ALTA	O sistema deve permitir vincular múltiplos veículos a um cliente
RF07	MEDIA	O sistema deve exibir histórico de atendimentos por cliente
RF08	ALTA	O sistema deve permitir busca de clientes por nome, CPF/CNPJ e placa
RF09	ALTA	O sistema deve permitir cadastro de veículos com dados completos
RF10	MEDIA	O sistema deve registrar quilometragem em cada atendimento
RF11	ALTA	O sistema deve permitir cadastro de mecânicos com especialidades
RF12	MEDIA	O sistema deve exibir histórico de OS por mecânico
RF13	ALTA	O sistema deve permitir cadastro de fornecedores com CNPJ
RF14	ALTA	O sistema deve permitir cadastro de produtos com código e preço
RF15	ALTA	O sistema deve vincular produtos a fornecedores
RF16	ALTA	O sistema deve registrar entrada de produtos via nota fiscal
RF17	ALTA	O sistema deve realizar baixa automática de estoque ao fechar OS
RF18	ALTA	O sistema deve emitir alertas quando estoque atingir nível mínimo

ID	Prioridade	Descrição
RF19	ALTA	O sistema deve exibir relatório de movimentação de estoque
RF20	MEDIA	O sistema deve calcular valor total do estoque em tempo real
RF21	ALTA	O sistema deve permitir criação de orçamentos com peças e serviços
RF22	ALTA	O sistema deve permitir aprovação ou rejeição de orçamentos
RF23	ALTA	O sistema deve converter orçamento aprovado em OS automaticamente
RF24	ALTA	O sistema deve registrar data e usuário de aprovação do orçamento
RF25	ALTA	O sistema deve criar OS com status, mecânico e itens
RF26	ALTA	O sistema deve permitir alteração de status da OS
RF27	ALTA	O sistema deve registrar peças e serviços na OS
RF28	MEDIA	O sistema deve calcular tempo total de execução da OS
RF29	ALTA	O sistema deve gerar documento de Nota Fiscal de Saída
RF30	ALTA	O sistema deve registrar forma de pagamento na NFS
RF31	ALTA	O sistema deve exibir relatório financeiro mensal e anual
RF32	ALTA	O sistema deve exibir dashboard com KPIs principais
RF33	ALTA	O sistema deve registrar log de todas as operações com timestamp
RF34	ALTA	O sistema deve identificar usuário responsável em cada operação
RF35	MEDIA	O sistema deve permitir consulta de auditoria por período e usuário
RF36	ALTA	O sistema deve realizar backup automático do banco de dados
RF37	MEDIA	O sistema deve permitir restauração manual do backup
RF38	ALTA	O sistema deve disponibilizar documentação Swagger da API
RF39	MEDIA	O sistema deve exibir relatório de produtividade por mecânico

ID	Prioridade	Descrição
RF40	BAIXA	O sistema deve permitir exportação de relatórios em PDF e Excel

6. Requisitos Não Funcionais (RNF)

ID	Categoria	Descrição
RNF01	Segurança	Todas as senhas devem ser criptografadas com BCrypt
RNF02	Segurança	Tokens JWT devem expirar em 1h (access) e 7d (refresh)
RNF03	Segurança	HTTPS obrigatório em produção
RNF04	Segurança	Proteção contra SQL Injection e XSS
RNF05	Desempenho	Respostas da API em até 2 segundos para 95% das requisições
RNF06	Desempenho	Sistema deve suportar 50 usuários simultâneos
RNF07	Disponibilidade	Uptime mínimo de 99% em horário comercial
RNF08	Manutenibilidade	Cobertura de testes mínima de 80% (JaCoCo)
RNF09	Manutenibilidade	Código aprovado no SonarQube (0 bugs críticos)
RNF10	Portabilidade	Aplicação containerizada com Docker
RNF11	Usabilidade	API documentada no Swagger/OpenAPI 3.0
RNF12	Escalabilidade	Arquitetura stateless para suporte a múltiplas instâncias

7. Regras de Negócio (RN)

ID	Módulo	Descrição
RN01	Usuários	Apenas ADMIN pode criar, editar e desativar usuários
RN02	Usuários	Não é permitido excluir usuário com histórico de operações
RN03	Clientes	CPF/CNPJ deve ser único no sistema
RN04	Clientes	Cliente inativo não pode abrir novo orçamento
RN05	Veículos	Placa deve ser única no sistema
RN06	Veículos	Veículo não pode ser excluído se possuir OS em aberto
RN07	Estoque	Não é permitido utilizar produto com estoque zero em OS
RN08	Estoque	Baixa de estoque só ocorre ao fechar a OS
RN09	Estoque	Alerta automático ao atingir estoque mínimo cadastrado
RN10	Orçamento	Orçamento rejeitado não pode ser convertido em OS
RN11	Orçamento	Orçamento aprovado não pode ser editado
RN12	Orçamento	Prazo de validade do orçamento é de 10 dias
RN13	OS	OS só pode ser aberta a partir de orçamento aprovado
RN14	OS	OS em andamento não pode ser excluída
RN15	OS	Ao cancelar OS, os itens de estoque são revertidos
RN16	OS	Apenas MECÂNICO pode alterar status para 'Em Andamento'
RN17	Faturamento	NFS só é gerada após OS com status 'Concluída'
RN18	Faturamento	Forma de pagamento é obrigatória ao emitir NFS
RN19	Auditoria	Todos os CRUDs geram registro de auditoria automático

ID	Módulo	Descrição
RN20	Backup	Backup automático executado diariamente às 02:00

8. Perfis de Acesso

ADMIN

- Acesso total ao sistema
- Gerenciar usuários e perfis
- Visualizar relatórios de auditoria
- Configurar parâmetros do sistema
- Executar e restaurar backup

GERENTE

- Acesso a todos os módulos exceto usuários
- Visualizar dashboard e relatórios
- Aprovar/rejeitar orçamentos
- Gerar Nota Fiscal de Saída
- Consultar log de auditoria

ATENDENTE

- Cadastrar e editar clientes e veículos
- Criar e editar orçamentos
- Abrir e editar OS
- Registrar entrada de produtos
- Emitir recibos

MECÂNICO

- Visualizar OS atribuídas
- Alterar status da OS
- Registrar peças e serviços na OS
- Consultar catálogo de produtos
- Visualizar histórico próprio

9. Fluxos de Negócio

9.1 Fluxo Completo de Atendimento

#	Etapa	Responsável
1	Cliente chega à oficina	ATENDENTE
2	Atendente cadastra/localiza o cliente	ATENDENTE
3	Cadastra/verifica o veículo	ATENDENTE
4	Cria orçamento com peças e serviços	ATENDENTE
5	Cliente aprova o orçamento	GERENTE
6	OS é gerada automaticamente	SISTEMA
7	Mecânico é atribuído à OS	GERENTE
8	Mecânico executa o serviço	MECÂNICO
9	Mecânico conclui a OS	MECÂNICO
10	Estoque é baixado automaticamente	SISTEMA
11	Atendente emite a Nota Fiscal	ATENDENTE
12	Cliente efetua pagamento	GERENTE

9.2 Fluxo de Controle de Estoque

#	Etapa	Responsável
1	Fornecedor entrega produtos	ATENDENTE
2	Nota Fiscal de Entrada é registrada	ATENDENTE
3	Estoque é atualizado automaticamente	SISTEMA
4	Produto é utilizado em OS	MECÂNICO
5	Baixa de estoque ao concluir OS	SISTEMA
6	Sistema verifica estoque mínimo	SISTEMA
7	Alerta enviado se abaixo do mínimo	SISTEMA

10. Modelo Conceitual das Entidades

Usuario

id
nome
email
senha (BCrypt)
perfil (ENUM)
ativo
createdAt
updatedAt

Cliente

id
nome
cpfCnpj
telefone
email
endereco
tipo (PF/PJ)
ativo

Veiculo

id
placa
marca
modelo
ano
cor
chassi
cliente_id

Mecanico

id
nome
cpf
especialidades
telefone
usuario_id
ativo

Fornecedor

id
razaoSocial
cnpj
telefone
email
endereco
ativo

Produto

id
codigo
descricao
categoria
preco_custo
preco_venda
estoque_atual
estoque_minimo
fornecedor_id

Orcamento

id

numero

cliente_id

veiculo_id

status (ENUM)

validade

total

createdBy

ItemOrcamento

id

orcamento_id

produto_id

servico_descricao

quantidade

preco_unitario

subtotal

OrdemServico

id

numero

orcamento_id

mecanico_id

status (ENUM)

dataAbertura

dataConclusao

observacoes

NotaFiscalSaida

id

numero

ordemServico_id

formaPagamento

valorTotal

dataEmissao

createdBy

NotaFiscalEntrada

id

numero

fornecedor_id

dataEntrada

valorTotal

itens

LogAuditoria

id

usuario_id

entidade

operacao

dadosAntigos

dadosNovos

timestamp

ip

11. Modelo de Banco de Dados

O banco de dados MySQL 8.x segue o modelo relacional normalizado (3FN). Abaixo estão os principais scripts DDL de criação das tabelas.

Tabela: usuarios

```
CREATE TABLE usuarios (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  senha       VARCHAR(255) NOT NULL,  
  perfil      ENUM('ADMIN', 'GERENTE', 'ATENDENTE', 'MECANICO') NOT NULL,  
  ativo       BOOLEAN DEFAULT TRUE,  
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at  DATETIME ON UPDATE CURRENT_TIMESTAMP  
);
```

Tabela: clientes

```
CREATE TABLE clientes (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nome        VARCHAR(150) NOT NULL,  
  cpf_cnpj    VARCHAR(18) NOT NULL UNIQUE,  
  tipo        ENUM('PF', 'PJ') NOT NULL,  
  telefone    VARCHAR(20),  
  email       VARCHAR(100),  
  logradouro  VARCHAR(200),  
  numero      VARCHAR(10),  
  bairro      VARCHAR(100),  
  cidade      VARCHAR(100),  
  uf          CHAR(2),  
  cep         VARCHAR(9),  
  ativo       BOOLEAN DEFAULT TRUE  
);
```

Tabela: veiculos

```
CREATE TABLE veiculos (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  placa       VARCHAR(8) NOT NULL UNIQUE,  
  marca       VARCHAR(50) NOT NULL,  
  modelo      VARCHAR(100) NOT NULL,  
  ano         YEAR NOT NULL,  
  cor         VARCHAR(30),  
  chassi      VARCHAR(17),  
  cliente_id  BIGINT NOT NULL,  
  FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

Tabela: produtos

```
CREATE TABLE produtos (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
codigo          VARCHAR(30) NOT NULL UNIQUE,  
descricao       VARCHAR(200) NOT NULL,  
categoria       VARCHAR(80),  
preco_custo     DECIMAL(10,2) NOT NULL,  
preco_venda     DECIMAL(10,2) NOT NULL,  
estoque_atual   INT DEFAULT 0,  
estoque_minimo  INT DEFAULT 5,  
fornecedor_id   BIGINT,  
ativo           BOOLEAN DEFAULT TRUE,  
FOREIGN KEY (fornecedor_id) REFERENCES fornecedores(id)  
);
```

Tabela: ordens_servico

```
CREATE TABLE ordens_servico (  
  id              BIGINT AUTO_INCREMENT PRIMARY KEY,  
  numero         VARCHAR(20) NOT NULL UNIQUE,  
  orcamento_id   BIGINT NOT NULL,  
  mecanico_id     BIGINT NOT NULL,  
  status          ENUM('ABERTA', 'EM_ANDAMENTO', 'CONCLUIDA', 'CANCELADA') NOT NULL,  
  data_abertura   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  data_conclusao  DATETIME,  
  observacoes     TEXT,  
  FOREIGN KEY (orcamento_id) REFERENCES orcamentos(id),  
  FOREIGN KEY (mecanico_id)   REFERENCES mecanicos(id)  
);
```

12. Estrutura das APIs REST

AUTH

POST	/api/auth/login	Autenticar usuário e retornar JWT
POST	/api/auth/refresh	Renovar access token com refresh token
POST	/api/auth/logout	Invalidar token do usuário

USUÁRIOS

GET	/api/usuarios	Listar todos os usuários (ADMIN)
POST	/api/usuarios	Criar novo usuário (ADMIN)
GET	/api/usuarios/{id}	Buscar usuário por ID
PUT	/api/usuarios/{id}	Atualizar dados do usuário
DELETE	/api/usuarios/{id}	Desativar usuário (soft-delete)

CLIENTES

GET	/api/clientes	Listar clientes com filtros
POST	/api/clientes	Cadastrar novo cliente
GET	/api/clientes/{id}	Buscar cliente por ID
PUT	/api/clientes/{id}	Atualizar cliente
DELETE	/api/clientes/{id}	Desativar cliente
GET	/api/clientes/{id}/veiculos	Listar veículos do cliente

VEÍCULOS

GET	/api/veiculos	Listar veículos com filtros
POST	/api/veiculos	Cadastrar veículo
GET	/api/veiculos/{id}	Buscar veículo por ID
PUT	/api/veiculos/{id}	Atualizar veículo

PRODUTOS / ESTOQUE

GET	/api/produtos	Listar produtos com estoque
POST	/api/produtos	Cadastrar produto
PUT	/api/produtos/{id}	Atualizar produto
GET	/api/produtos/alerta	Listar produtos abaixo do mínimo
POST	/api/nf-entrada	Registrar NF de entrada
GET	/api/estoque/movimentacao	Relatório de movimentação

ORÇAMENTOS / OS

GET	/api/orcamentos	Listar orçamentos
POST	/api/orcamentos	Criar orçamento
PUT	/api/orcamentos/{id}/aprovar	Aprovar orçamento
PUT	/api/orcamentos/{id}/rejeitar	Rejeitar orçamento
GET	/api/ordens-servico	Listar OS com filtros
PUT	/api/ordens-servico/{id}/status	Atualizar status da OS
POST	/api/nf-saida	Emitir NF de saída

RELATÓRIOS

GET	/api/relatorios/financeiro	Relatório financeiro por período
GET	/api/relatorios/mecanicos	Produtividade por mecânico
GET	/api/dashboard	KPIs do dashboard
GET	/api/auditoria	Log de auditoria por filtros

13. Estrutura do Projeto Spring Boot

Arquitetura em camadas seguindo padrões MVC + Service Layer + Repository Pattern:

```
com.oficina
├── config/
│   ├── SecurityConfig.java           # Spring Security + JWT Filter
│   ├── SwaggerConfig.java           # OpenAPI 3.0 configuração
│   ├── AuditConfig.java              # Configuração de auditoria
│   └──
├── controller/
│   ├── AuthController.java           # Login, refresh, logout
│   ├── UsuarioController.java
│   ├── ClienteController.java
│   ├── VeiculoController.java
│   ├── MecanicoController.java
│   ├── FornecedorController.java
│   ├── ProdutoController.java
│   ├── EstoqueController.java
│   ├── OrcamentoController.java
│   ├── OrdemServicoController.java
│   ├── FaturamentoController.java
│   ├── RelatorioController.java
│   └── AuditoriaController.java
│   └──
├── service/
│   ├── AuthService.java
│   ├── ClienteService.java           # Regras de negócio de clientes
│   ├── EstoqueService.java           # Baixa e alertas de estoque
│   ├── OrdemServicoService.java      # Fluxo completo da OS
│   └── ...                            # (um service por módulo)
│   └──
├── repository/
│   └── ...                            # Interfaces JPA/Hibernate
│   └──
├── model/                             # Entidades JPA (@Entity)
│   ├── Usuario.java
│   ├── Cliente.java
│   └── ...
│   └──
├── dto/                               # Data Transfer Objects
│   ├── request/
│   └── response/
│   └──
├── mapper/                            # MapStruct mappers
│   └──
├── security/
│   ├── JwtUtil.java                  # Geração e validação JWT
│   ├── JwtFilter.java                # Filtro de autenticação
│   └── UserDetailsServiceImpl.java
│   └──
├── exception/
│   ├── GlobalExceptionHandler.java  # @ControllerAdvice
│   └── BusinessException.java
```

```
■ ■■■ ResourceNotFoundException.java
■
■■■ OficinaMecanicaApplication.java
```

13.1 Dependências Maven (pom.xml)

Dependência	Finalidade
spring-boot-starter-web	API REST
spring-boot-starter-data-jpa	Persistência com Hibernate
spring-boot-starter-security	Segurança
spring-boot-starter-validation	Validação de DTOs
mysql-connector-j	Driver MySQL
jjwt-api / jjwt-impl	Geração e validação JWT
mapstruct	Mapeamento Entity ↔ DTO
springdoc-openapi-starter-webmvc-ui	Swagger UI
lombok	Redução de boilerplate
spring-boot-starter-test	Testes unitários
mockito-core	Mocks para testes
jacoco-maven-plugin	Cobertura de testes

14. Tecnologias Utilizadas

BACKEND

Tecnologia	Finalidade
Java 17+	Linguagem principal (LTS)
Spring Boot 3.x	Framework principal
Spring Security	Segurança e autenticação
Spring Data JPA	Persistência de dados
Hibernate	ORM para mapeamento objeto-relacional
JWT (JJWT)	Tokens de autenticação stateless
MapStruct	Mapeamento automático Entity/DTO
Lombok	Redução de código boilerplate
Maven	Gerenciamento de dependências e build

BANCO DE DADOS

Tecnologia	Finalidade
MySQL 8.x	Banco de dados relacional principal
Flyway	Controle de versão do schema
H2 Database	Banco em memória para testes

QUALIDADE E TESTES

Tecnologia	Finalidade
JUnit 5	Framework de testes unitários
Mockito	Mocks e stubs para testes
JaCoCo	Cobertura de código (meta: 80%+)
SonarQube	Análise estática de qualidade
AssertJ	Assertions fluentes nos testes

DOCUMENTAÇÃO E DEVOPS

Tecnologia	Finalidade
Swagger / OpenAPI 3.0	Documentação automática da API
Docker	Containerização da aplicação

Tecnologia	Finalidade
Docker Compose	Orquestração local (app + MySQL)
Git + GitHub	Controle de versão
GitHub Actions	CI/CD pipeline

15. Estratégia de Testes

A estratégia de testes segue a pirâmide de testes, com foco em testes unitários e de integração, atingindo cobertura mínima de 80% mensurada pelo JaCoCo.

15.1 Testes Unitários (JUnit 5 + Mockito)

Classe de Teste	Cenários Cobertos
UsuarioServiceTest	Criação, edição, desativação de usuários
ClienteServiceTest	Validação CPF/CNPJ, soft-delete, busca
EstoqueServiceTest	Baixa de estoque, alertas, movimentação
OrcamentoServiceTest	Criação, aprovação, rejeição de orçamentos
OrdemServicoServiceTest	Abertura, status, conclusão, cancelamento
FaturamentoServiceTest	Geração de NFS, formas de pagamento
JwtUtilTest	Geração, validação e expiração de tokens

15.2 Exemplo de Teste Unitário

```
@ExtendWith(MockitoExtension.class)
class ClienteServiceTest {

    @Mock
    private ClienteRepository clienteRepository;

    @InjectMocks
    private ClienteService clienteService;

    @Test
    @DisplayName("Deve lançar exceção ao cadastrar CPF duplicado")
    void deveLancarExcecao_QuandoCpfJaExiste() {
        // Given
        ClienteRequestDTO dto = new ClienteRequestDTO("João", "123.456.789-00", ...);
        when(clienteRepository.existsByCpfCnpj("123.456.789-00")).thenReturn(true);

        // When / Then
        assertThrows(BusinessException.class,
            () -> clienteService.cadastrar(dto));
        verify(clienteRepository, never()).save(any());
    }
}
```

15.3 Configuração JaCoCo (pom.xml)

```
org.jacoco
jacoco-maven-plugin
```

BUNDLE

LINE
COVEREDRATIO
0.80

16. Casos de Uso Principais

UC01 — Autenticar no Sistema

Ator Principal:	Todos os usuários
Fluxo Principal:	1. Usuário acessa a tela de login 2. Informa e-mail e senha 3. Sistema valida credenciais 4. Sistema retorna JWT (access + refresh token) 5. Usuário é redirecionado ao dashboard
Exceções:	Credenciais inválidas → mensagem de erro; conta inativa → acesso bloqueado

UC02 — Criar Ordem de Serviço

Ator Principal:	ATENDENTE / GERENTE
Fluxo Principal:	1. Atendente localiza o orçamento aprovado 2. Sistema gera OS automaticamente 3. Gerente atribui mecânico à OS 4. OS fica com status ABERTA 5. Mecânico é notificado
Exceções:	Orçamento não aprovado → não permite gerar OS; mecânico inativo → não aparece na lista

UC03 — Concluir Ordem de Serviço

Ator Principal:	MECÂNICO
Fluxo Principal:	1. Mecânico seleciona a OS em andamento 2. Registra peças utilizadas e serviços 3. Altera status para CONCLUÍDA 4. Sistema realiza baixa automática no estoque 5. OS fica disponível para faturamento
Exceções:	Estoque insuficiente → OS não pode ser concluída; peça obrigatória não informada → validação

UC04 — Emitir Nota Fiscal de Saída

Ator Principal:	ATENDENTE / GERENTE
Fluxo Principal:	1. Seleciona OS com status CONCLUÍDA 2. Informa forma de pagamento 3. Sistema calcula valor total 4. Gera NFS com número sequencial 5. Cliente recebe comprovante
Exceções:	OS não concluída → NFS bloqueada; forma de pagamento não informada → validação

17. Roadmap de Sprints

Projeto organizado em 10 Sprints de 2 semanas, totalizando 364 horas de desenvolvimento. Metodologia Scrum adaptada para desenvolvedor solo.

Sprint 1	Segurança e Autenticação	40h
<i>Semanas 1-2</i>	Configuração do projeto Spring Boot • Spring Security + JWT • Cadastro de usuários • Controle de perfis (ADMIN, GERENTE, ATENDENTE, MECÂNICO) • BCrypt para senhas • Testes unitários da camada de segurança	
Sprint 2	Clientes e Veículos	24h
<i>Semanas 3-4</i>	CRUD completo de Clientes (PF e PJ) • CRUD completo de Veículos • Validação de CPF/CNPJ e Placa • Busca e filtros • Testes de service	
Sprint 3	Mecânicos, Fornecedores e Produtos	32h
<i>Semanas 5-6</i>	CRUD de Mecânicos com especialidades • CRUD de Fornecedores (CNPJ) • Catálogo de Produtos • Vinculação Produto-Fornecedor • Testes unitários	
Sprint 4	Estoque e NF de Entrada	40h
<i>Semanas 7-8</i>	Registro de Nota Fiscal de Entrada • Atualização automática de estoque • Alertas de estoque mínimo • Relatório de movimentação • Testes de integração	
Sprint 5	Orçamento e Ordem de Serviço	60h
<i>Semanas 9-10</i>	Criação e gestão de Orçamentos • Fluxo de aprovação/rejeição • Geração automática de OS • Gestão de status da OS • Baixa automática de estoque • Testes de fluxo completo — cenário mais crítico	
Sprint 6	Faturamento e NF de Saída	40h
<i>Semanas 11-12</i>	Emissão de NF de Saída • Formas de pagamento • Geração de recibo para cliente • Relatório financeiro mensal/anual • Testes	
Sprint 7	Dashboard e Relatórios	32h
<i>Semanas 13-14</i>	Dashboard com KPIs principais • Relatório de produtividade por mecânico • Relatório de estoque • Exportação PDF/Excel • Testes de relatórios	
Sprint 8	Auditoria e Backup	32h
<i>Semanas 15-16</i>	Log automático de todas as operações • Consulta de auditoria por filtros • Backup automático agendado (cron) • Restauração de backup • Testes	

Sprint 9	Testes, JaCoCo, SonarQube e Swagger	40h
<i>Semanas 17-18</i>	Cobertura mínima de 80% com JaCoCo • Análise SonarQube (0 bugs críticos) • Documentação Swagger/OpenAPI 3.0 completa • Ajustes de qualidade de código	
Sprint 10	Homologação e Deploy	24h
<i>Semanas 19-20</i>	Testes de homologação end-to-end • Containerização com Docker + Docker Compose • Pipeline CI/CD com GitHub Actions • Deploy em servidor • Documentação final	

Área	Horas	%
Backend (Entities, Services, Controllers)	220h	60%
Segurança (Spring Security + JWT)	40h	11%
Banco de Dados (Modelagem + Scripts)	24h	7%
Testes (JUnit + Mockito + JaCoCo)	40h	11%
Documentação (Swagger + UML)	20h	5%
Qualidade (SonarQube + Refatoração)	20h	5%
TOTAL	364h	100%

18. Plano de Backup e Recuperação

18.1 Estratégia de Backup

Tipo	Horário	Descrição	Retenção
Diário	02:00 AM	Full backup do banco MySQL	7 dias
Semanal	Domingo 03:00	Backup compactado (.gz)	4 semanas
Mensal	Dia 1 às 04:00	Backup arquivado completo	12 meses

18.2 Script de Backup Automático

```
#!/bin/bash
# backup_oficina.sh - Executado via cron

DB_NAME="oficina_mecanica"
DB_USER="backup_user"
DB_PASS="senha_segura"
BACKUP_DIR="/backups/oficina"
DATE=$(date +%Y%m%d_%H%M%S)
FILE="$BACKUP_DIR/backup_$DATE.sql.gz"

# Criar backup compactado
mysqldump -u$DB_USER -p$DB_PASS $DB_NAME | gzip > $FILE

# Remover backups com mais de 7 dias
find $BACKUP_DIR -name "*.sql.gz" -mtime +7 -delete

echo "Backup concluído: $FILE" >> /var/log/backup_oficina.log
```

18.3 Configuração Crontab

```
0 2 * * * /scripts/backup_oficina.sh
```

Procedimento de Restauração: Em caso de falha, executar `gunzip backup_YYYYMMDD.sql.gz | mysql -u root -p oficina_mecanica`. Tempo estimado de restauração: menos de 10 minutos para bases de até 1GB.

19. Glossário

API REST	Interface de programação que utiliza protocolo HTTP e arquitetura REST
BCrypt	Algoritmo de hash seguro para armazenamento de senhas
DTO	Data Transfer Object — objeto usado para transferir dados entre camadas
Flyway	Ferramenta de versionamento de schema de banco de dados
JaCoCo	Biblioteca Java para análise de cobertura de código
JPA	Java Persistence API — especificação para persistência de dados
JWT	JSON Web Token — padrão para autenticação stateless
MapStruct	Processador de anotações para mapeamento automático entre objetos
Mockito	Framework Java para criação de objetos mock em testes
NF de Entrada	Nota Fiscal registrada ao receber produtos de fornecedores
NF de Saída	Nota Fiscal emitida ao cliente ao concluir serviço
OpenAPI	Especificação para documentação de APIs REST
OS	Ordem de Serviço — documento que registra o trabalho a ser executado
Soft-delete	Inativação de registro sem exclusão física do banco
SonarQube	Plataforma de análise estática de qualidade de código
Spring Boot	Framework Java para criação de aplicações standalone
Spring Security	Módulo do Spring para controle de autenticação e autorização
Swagger	Interface visual para exploração e teste de APIs REST